

1. Постановка задачи и данные

Цель проекта — проверить, насколько хорошо можно предсказать победителя матча League of Legends по игровой статистике и как качество прогноза меняется в зависимости от доступной информации о матче.

В работе используется датасет League of Legends Ranked Games. Обучающая выборка содержит 40 842 матча и 61 исходный столбец. Целевая переменная winner принимает значения 1 и 2 и показывает победившую команду.

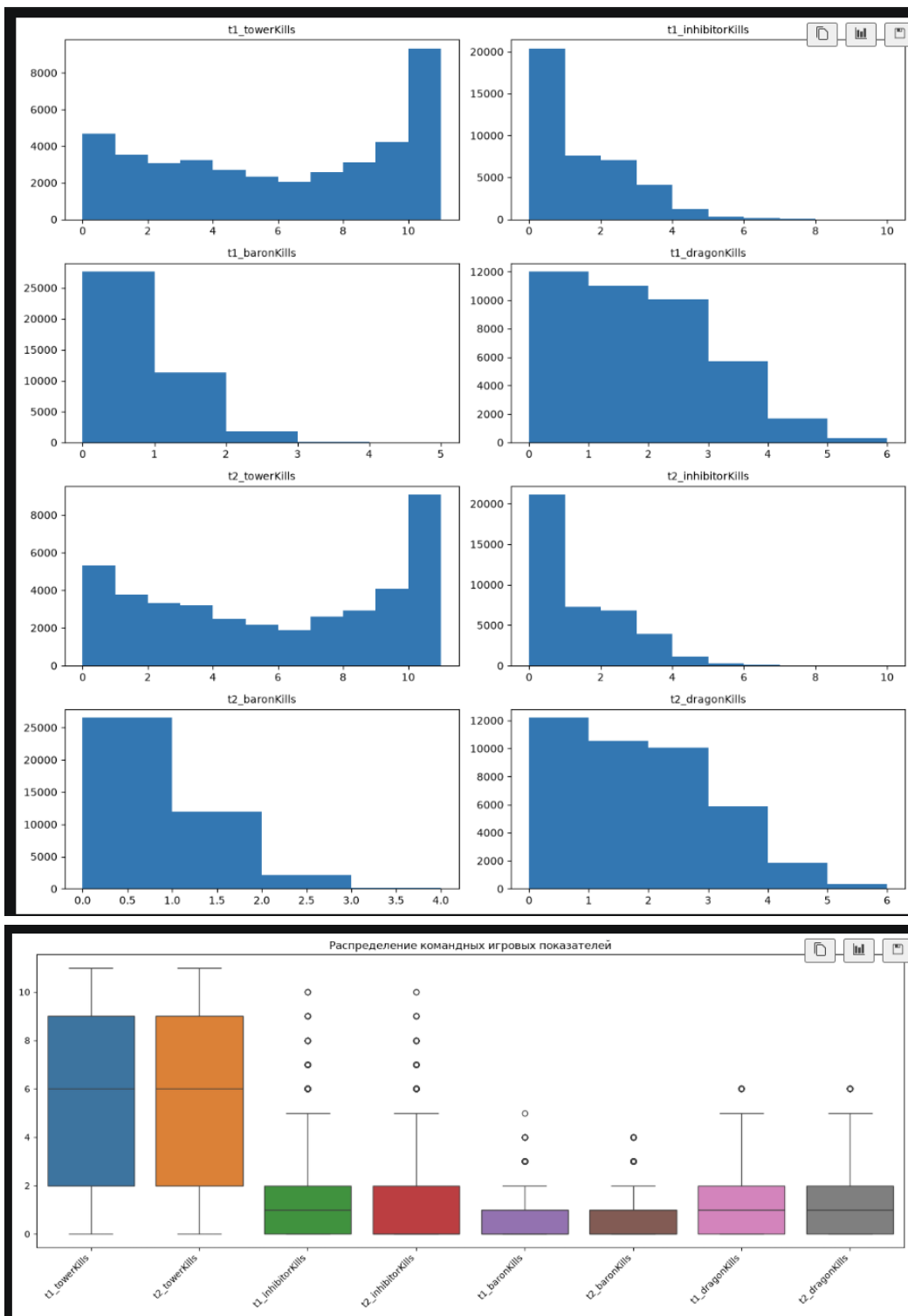
Источник данных: https://www.kaggle.com/datasnaek/league-of-legends/data?select=summoner_spell_info.json

Основная гипотеза: по одному только драфту результат матча должен предсказываться слабо, а после появления информации о ходе игры качество должно заметно расти. Поэтому данные разделены на три набора: draft, early_game и full.

2. EDA и предобработка

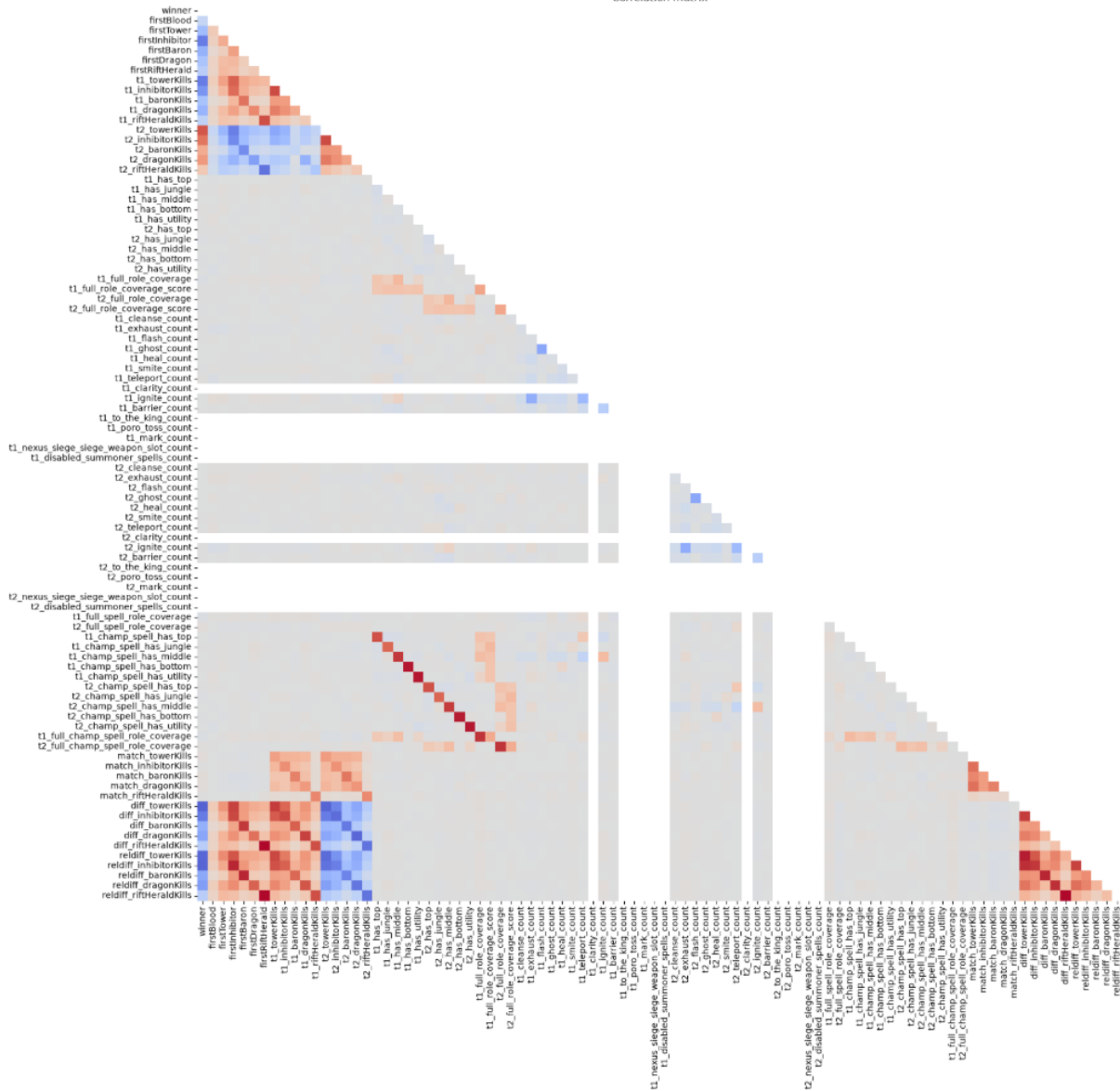
Пропусков в данных нет. Были удалены дубликаты, пустые и константные признаки. gameId, creationTime, seasonId и gameDuration исключены из рабочих наборов, так как не используются как содержательные признаки.

Распределения командных игровых показателей выглядят логично. Для ингибиторов, баронов и драконов встречаются редкие высокие значения, которые формально считаются выбросами по IQR, но остаются допустимыми игровыми результатами, поэтому их не удаляю.



По корреляционной матрице сильнее всего с победителем связаны признаки, которые непосредственно описывают ход матча: первые объекты, башни, ингибиторы, драконы и другие цели. Пики и баны по отдельности имеют слабую линейную связь с winner, что ожидаемо из-за высокой разреженности таких признаков.

Correlation matrix



3. Feature engineering

Для исходных ID чемпионов, банов и заклинаний были созданы более интерпретируемые признаки. Для определения ролей чемпионов использовался проект Meraki Analytics Role Identification, так как исходный датасет не содержал нужного разбиения по ролям.

Использован commit 2d10b01: <https://github.com/meraki-analytics/role-identification/tree/2d10b01ebaf27af3672fb6abe36cd6beeb22796d>

- отдельные признаки пика и бана каждого героя с указанием команды;
- количество конкретных заклинаний у каждой команды;
- наличие и покрытие игровых ролей;
- соответствие заклинаний предполагаемым ролям;
- суммы, разности и относительные разности командных игровых показателей.

Были проверены полный набор feature_max и сокращённый feature_min. Feature_min формально оказался чуть лучше, но разница меньше 0.001 F1 и находится внутри разброса между фолдами. Далее используется feature_max, так как он сохраняет весь созданный набор признаков.

4. Разделение данных по этапам матча

Набор	Что содержит	Смысл
draft	Пики, баны, роли, заклинания	Информация до начала матча
early_game	Draft + первые игровые события	Ход игры без итоговых счётчиков
full	Полный рабочий набор	Максимально доступная информация

Такое разделение позволяет проверить, что сильнее влияет на качество: выбор алгоритма или объём информации, который уже известен о матче.

5. Классические модели

В качестве baseline использованы Logistic Regression и Decision Tree. Далее проверены CatBoost, XGBoost и Random Forest. Гиперпараметры подбирались на внутренних фолдах, итоговое качество оценивалось на внешней 5-fold кросс-валидации. Основная метрика — F1 macro.

Набор	LogReg F1	Decision Tree F1
full	0.9690	0.9680
early_game	0.8935	0.8955
draft	0.5485	0.5072

Набор	CatBoost F1	XGBoost F1	Random Forest F1
full	0.9702	0.9705	0.9701
early_game	0.8986	0.9004	0.8997
draft	0.5416	0.5454	0.5424

На full все сильные модели дают качество около $F1 = 0.97$, на early_game — около 0.90, а на draft — только около 0.54–0.55. Лучший результат на draft остаётся у Logistic Regression, на early_game и full — у XGBoost.

По времени XGBoost оказался наиболее удобным ансамблем. На full nested CV занял около 379 секунд против 1891 у CatBoost и 1609 у Random Forest. На early_game — около 321 секунды против 1867 и 2986 соответственно.

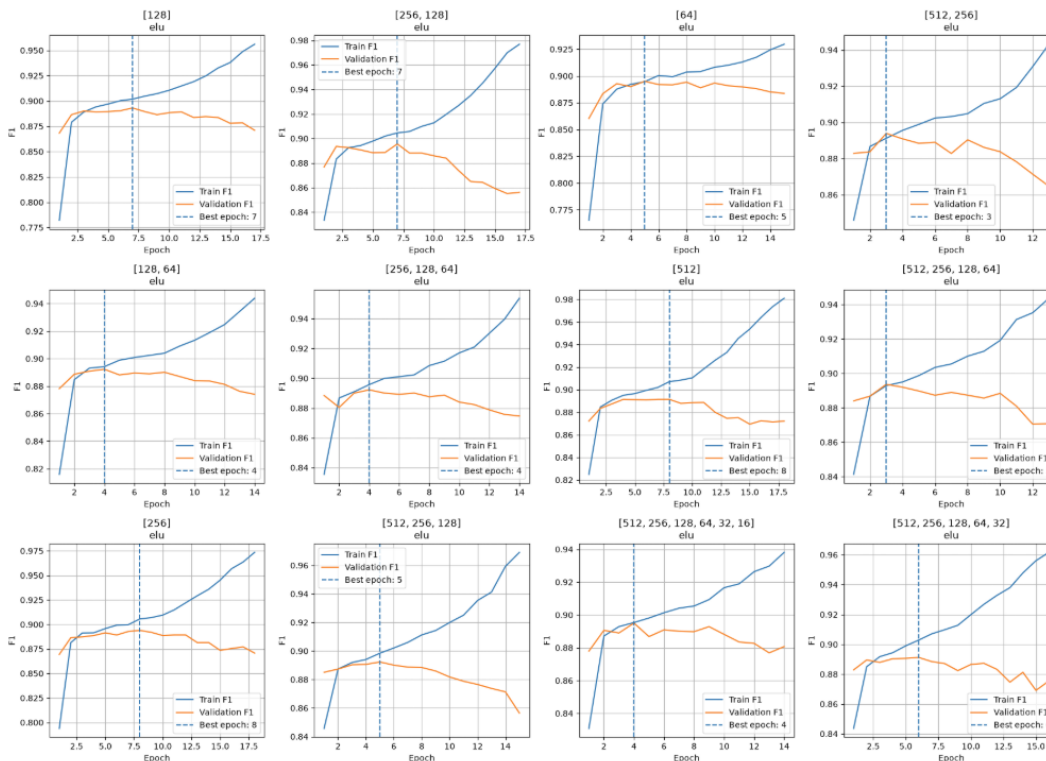
6. Многослойный перцептрон

Для нейросетевой модели используется MLP с бинарным выходом и BCEWithLogitsLoss. Признаки стандартизируются только по обучающей части соответствующего фолда. Подбор проводился последовательно: архитектура и активация, регуляризация, затем learning rate и batch size.

6.1. Архитектура и функция активации

Архитектура	Активация	F1 mean	F1 std
[128]	ELU	0.8879	0.0026
[256, 128]	ELU	0.8875	0.0037
[64]	ELU	0.8872	0.0021
[512, 256]	ELU	0.8863	0.0030
[128, 64]	ELU	0.8861	0.0024

Лучший средний результат показала простая архитектура [128] с ELU. Более глубокие сети стабильного преимущества не дали, а ELU заняла верхние позиции почти для всех архитектур.



6.2. Регуляризация и оптимизация

Лучшей конфигурацией регуляризации стала [128] + ELU + Dropout 0.8 + weight decay 1e-4: F1 = 0.8914 ± 0.0023. После добавления регуляризации nested

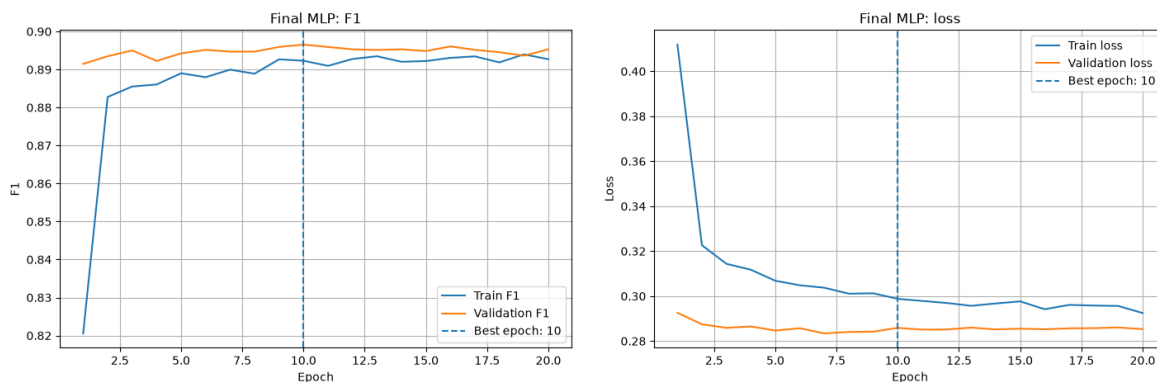
CV вырос с 0.8883 ± 0.0026 до 0.8926 ± 0.0005 . Основной эффект дал Dropout = 0.8, влияние weight decay оказалось небольшим.

При подборе параметров оптимизации лучший средний результат получен при learning rate = 0.001 и batch size = 128: F1 = 0.8914. Разница между проверенными комбинациями небольшая, поэтому на этом этапе заметного дополнительного прироста уже нет.

6.3. Итоговая MLP

Набор	Accuracy	F1	ROC-AUC	Время, с
draft	0.5458	0.5444	0.5682	213
early_game	0.8933	0.8933	0.9429	245
full	0.9675	0.9675	0.9969	395

MLP повторяет общую картину классических моделей, но не выигрывает ни на одном этапе. На early_game XGBoost лучше примерно на 0.007 F1, на full — примерно на 0.003.



На выбранном фолде лучшая эпоха — около 10. Validation F1 и validation loss после неё остаются почти стабильными, сильного разрыва между train и validation нет. Выявленного переобучения по этим кривым не видно.

7. Сравнение всех моделей

Ниже приведена общая таблица результатов. RMSE считается по hard labels и добавлен из-за требования задания. Время обучения используется как ориентир вычислительной стоимости: для классических моделей оно включает nested CV с подбором параметров, а для итоговой MLP — оценку уже зафиксированной конфигурации.

Набор	Модель	Accuracy	F1	RMSE	ROC-AUC	Время, с
draft	Logistic Regression	0.5495	0.5485	0.6712	0.5721	321
draft	XGBoost	0.5473	0.5454	0.6729	0.5662	264
draft	MLP	0.5458	0.5444	0.6739	0.5682	213

draft	Random Forest	0.5426	0.5424	0.6763	0.5605	2651
draft	CatBoost	0.5441	0.5416	0.6752	0.5643	1891
draft	Decision Tree	0.5130	0.5072	0.6979	0.5185	34
early_game	XGBoost	0.9005	0.9004	0.3155	0.9580	321
early_game	Random Forest	0.8998	0.8997	0.3166	0.9565	2986
early_game	CatBoost	0.8986	0.8986	0.3184	0.9569	1867
early_game	Decision Tree	0.8955	0.8955	0.3232	0.9537	23
early_game	Logistic Regression	0.8936	0.8935	0.3262	0.9435	183
early_game	MLP	0.8933	0.8933	0.3266	0.9429	245
full	XGBoost	0.9705	0.9705	0.1718	0.9974	379
full	CatBoost	0.9702	0.9702	0.1725	0.9974	1891
full	Random Forest	0.9702	0.9701	0.1728	0.9971	1609
full	Logistic Regression	0.9690	0.9690	0.1761	0.9971	250
full	Decision Tree	0.9680	0.9680	0.1788	0.9950	19
full	MLP	0.9675	0.9675	0.1802	0.9969	395

Итоговый выбор по CV: Logistic Regression для draft, XGBoost для early_game и full. Разница между сильными моделями внутри одного этапа обычно составляет тысячные доли F1, тогда как переход от draft к early_game даёт прирост примерно на 0.35 F1.

8. Финальная проверка на тестовой выборке

Тестовая выборка до финального этапа не использовалась для выбора признаков, моделей или гиперпараметров. После фиксации победителей каждая модель была один раз обучена на всём соответствующем train-наборе и проверена на untouched test.

Этап	Модель	Accuracy	F1	RMSE	ROC-AUC	Время, с
draft	Logistic Regression	0.5507	0.5500	0.6703	0.5697	3.7
early_game	XGBoost	0.8987	0.8987	0.3182	0.9597	2.7
full	XGBoost	0.9711	0.9711	0.1700	0.9976	3.6

Test практически полностью подтвердил CV: разница по F1 составила +0.0015 для draft, -0.0017 для early_game и +0.0006 для full. Все отклонения меньше 0.002, поэтому оценка получилась стабильной.

9. Итоговый вывод

Основная гипотеза проекта подтвердилась. По одному draft исход матча предсказывается слабо: $F1 = 0.5500$ и $ROC-AUC = 0.5697$. После появления первых игровых событий качество резко возрастает до $F1 = 0.8987$ и $ROC-AUC = 0.9597$. На полном наборе статистики результат достигает $F1 = 0.9711$ и $ROC-AUC = 0.9976$.

Финальными моделями стали Logistic Regression для draft и XGBoost для early_game и full. MLP не смогла превзойти лучшие классические алгоритмы. Поэтому в этой задаче основное влияние на качество оказывает не сложность модели, а объём информации о ходе матча.